

Products Exercise Requirements Checklist

Advanced .NET Exercise: Product Management API

Module 1: Advanced AutoMapper Patterns for Products (20 minutes)

Task 1.1: Create Advanced Product Profile (8 minutes)

Step 1: Enhanced Entities and DTOs

Requirements:

- ☐ Create `ProductCategory` enum with values: Electronics (0), Clothing (1), Books (2), Home (3)
- ☐ Update `Product` entity to include:
 - ☐ `string Name` property
 - ☐ `string Brand` property
 - ☐ `string SKU` property
 - ☐ `ProductCategory Category` property
 - ☐ `decimal Price` property
 - ☐ `DateTime ReleaseDate` property
 - ☐ `string? ImageUrl` property (optional)
 - ☐ `bool IsAvailable` property (default based on stock)
 - ☐ `int StockQuantity` property (default 0)
- ☐ Create `ProductProfileDto` with properties:
 - ☐ `Guid Id`
 - ☐ `string Name`
 - ☐ `string Brand`
 - ☐ `string SKU`
 - ☐ `string CategoryDisplayName`
 - ☐ `decimal Price`
 - ☐ `string FormattedPrice`
 - ☐ `DateTime ReleaseDate`
 - ☐ `DateTime CreatedAt`
 - ☐ `string? ImageUrl`

- [] bool IsAvailable
- [] int StockQuantity
- [] string ProductAge
- [] string BrandInitials
- [] string AvailabilityStatus
- [] Create `CreateProductProfileRequest` with properties:
 - [] string Name
 - [] string Brand
 - [] string SKU
 - [] `ProductCategory` Category
 - [] decimal Price
 - [] DateTime ReleaseDate
 - [] string? ImageUrl (optional)
 - [] int StockQuantity (default 1)

Step 2: Advanced Product Mapping Profile

Requirements:

- [] Create `AdvancedProductMappingProfile` class inheriting from `Profile`
- [] Map `CreateProductProfileRequest` to `Product` with:
 - [] Auto-generated `Id` (`Guid.NewGuid()`)
 - [] Auto-generated `CreatedAt` (`DateTime.UtcNow`)
 - [] Auto-set `IsAvailable` based on `StockQuantity > 0`
 - [] Ignore `UpdatedAt`
- [] Map `Product` to `ProductProfileDto` using custom resolvers:
 - [] `CategoryDisplayResolver` for category display names
 - [] `PriceFormatterResolver` for currency formatting
 - [] `ProductAgeResolver` for human-readable product age
 - [] `BrandInitialsResolver` for brand initials
 - [] `AvailabilityStatusResolver` for stock status

Custom Value Resolvers Required:

- [] **CategoryDisplayResolver:**
 - Electronics → "Electronics & Technology"
 - Clothing → "Clothing & Fashion"
 - Books → "Books & Media"
 - Home → "Home & Garden"
 - Default → "Uncategorized"
- [] **PriceFormatterResolver:**
 - Format as currency using `ToString("C2")`
- [] **ProductAgeResolver:**
 - < 30 days → "New Release"
 - < 365 days → "X months old"

- < 1825 days (5 years) → "X years old"
- = 1825 days → "Classic"
- [] **BrandInitialsResolver:**
 - Two+ words → First letter of first and last words (uppercase)
 - Single word → First letter (uppercase)
 - No brand → "?"
- [] **AvailabilityStatusResolver:**
 - Not available → "Out of Stock"
 - Available, 0 stock → "Unavailable"
 - Available, 1 stock → "Last Item"
 - Available, ≤5 stock → "Limited Stock"
 - Available, >5 stock → "In Stock"

Task 1.2: Conditional Mapping Based on Product Category (7 minutes)

Requirements:

- [] Conditional **ImageUrl** mapping:
 - [] Return actual URL for Electronics, Clothing, Books categories
 - [] Return null for Home category (content filtering)
- [] Conditional **Price** mapping:
 - [] Apply 10% discount for Home category (Price * 0.9m)
 - [] Return actual price for all other categories

Task 1.3: Update CreateProductHandler (5 minutes)

Requirements:

- [] Update method signature to accept **CreateProductProfileRequest**
- [] Update logging to include Name, Brand, Category, SKU information
- [] Check SKU uniqueness instead of email uniqueness
- [] Use advanced mapping for Product creation
- [] Return **ProductProfileDto** instead of basic response
- [] Update cache key to "all_products"
- [] Maintain existing functionality (validation, caching, exception handling)

Module 2: Structured Logging & Telemetry for Products (20 minutes)

Task 2.1: Create Product-Specific Logging Infrastructure (8 minutes)

LogEvents Constants Required:

- [] ProductCreationStarted = 2001
- [] ProductValidationFailed = 2002
- [] ProductCreationCompleted = 2003
- [] DatabaseOperationStarted = 2004
- [] DatabaseOperationCompleted = 2005
- [] CacheOperationPerformed = 2006
- [] SKUValidationPerformed = 2007
- [] StockValidationPerformed = 2008

ProductCreationMetrics Record Required:

- [] string OperationId
- [] string ProductName
- [] string SKU
- [] ProductCategory Category
- [] TimeSpan ValidationDuration
- [] TimeSpan DatabaseSaveDuration
- [] TimeSpan TotalDuration
- [] bool Success
- [] string? ErrorReason (optional)

LoggingExtensions Required:

- [] Update LogProductCreationMetrics() method:
 - [] Log product-specific metrics with event ID
 - [] Include Name, SKU, Category in log message
 - [] Include all timing measurements in milliseconds
 - [] Include success/failure status

Task 2.2: Implement Performance Tracking for Products (7 minutes)

Requirements:

- [] Track operation start time for product creation
- [] Generate unique operation ID (8 characters)
- [] Use logging scope for entire product operation
- [] Log operation start with Name, Brand, SKU, Category details
- [] Add SKU validation logging with SKUValidationPerformed event
- [] Add stock validation logging with StockValidationPerformed event
- [] Time validation phase separately (SKU uniqueness check)
- [] Log validation failures with product-specific details
- [] Time database operations separately
- [] Log database operation start and completion with ProductId
- [] Log cache operations with "all_products" cache key
- [] Calculate total operation duration
- [] Log comprehensive ProductCreationMetrics for success cases

- ☐ Log error metrics in catch block with product details
- ☐ Re-throw exceptions for global handler

Task 2.3: Correlation ID Middleware (5 minutes)

Requirements:

- ☐ Use same CorrelationMiddleware as user exercise
 - ☐ Ensure correlation IDs flow through product operations
 - ☐ Test correlation with product-specific logging
-

Module 3: Complex Product Validation Scenarios (25 minutes)

Task 3.1: Create Advanced Product Validation Infrastructure (10 minutes)

CreateProductProfileValidator Requirements:

- ☐ Inject `ApplicationContext` and `ILogger<CreateProductProfileValidator>`
- ☐ **Name Validation Rules:**
 - ☐ Not empty with custom message
 - ☐ Minimum 1 character, maximum 200 characters
 - ☐ Must not contain inappropriate content (validate against word list)
 - ☐ Must be unique for the same brand (async database check)
- ☐ **Brand Validation Rules:**
 - ☐ Not empty with custom message
 - ☐ Minimum 2 characters, maximum 100 characters
 - ☐ Must contain only valid characters (letters, spaces, hyphens, apostrophes, dots, numbers)
- ☐ **SKU Validation Rules:**
 - ☐ Not empty with custom message
 - ☐ Must be valid SKU format (alphanumeric with hyphens, 5-20 characters)
 - ☐ Must be unique in system (async database check)
- ☐ **Category Validation Rules:**
 - ☐ Must be valid enum value
- ☐ **Price Validation Rules:**
 - ☐ Must be greater than 0
 - ☐ Must be less than \$10,000
- ☐ **ReleaseDate Validation Rules:**
 - ☐ Cannot be in the future
 - ☐ Cannot be before year 1900
- ☐ **StockQuantity Validation Rules:**
 - ☐ Cannot be negative
 - ☐ Cannot exceed 100,000 (reasonableness check)

- [] **ImageUrl Validation Rules:**
 - [] Must be valid image URL when provided
 - [] Must be HTTP/HTTPS protocol
 - [] Must end with image extensions (.jpg, .jpeg, .png, .gif, .webp)
- [] **Business Rules Validation:**
 - [] Must pass all complex business rules (async)

Validation Methods Required:

- [] `BeValidName()`: Check against inappropriate words list
- [] `BeUniqueName()`: Async database check for Name+brand combination with logging
- [] `BeValidBrandName()`: Regex validation for allowed characters
- [] `BeValidSKU()`: Format validation (alphanumeric with hyphens)
- [] `BeUniqueSKU()`: Async database check with logging
- [] `BeValidImageUrl()`: URL and image extension validation
- [] `PassBusinessRules()`: Complex async business rule validation

Business Rules Required:

- [] **Rule 1:** Daily product addition limit check (max 500 per day)
- [] **Rule 2:** Electronics minimum price check (\$50.00)
- [] **Rule 3:** Home product content restrictions (check Name against restricted words)
- [] **Rule 4:** High-value product stock limit (>\$500 = max 10 stock)
- [] All rules must include appropriate logging

Task 3.2: Create Custom Product Validation Attributes (8 minutes)

ValidSKUAttribute Requirements:

- [] Inherit from `ValidationAttribute` and implement `IClientModelValidator`
- [] Implement `IsValid()` method to validate SKU format (alphanumeric with hyphens, 5-20 chars)
- [] Remove spaces before validation
- [] Implement `AddValidation()` for client-side validation
- [] Add data attributes for client SKU validation

ProductCategoryAttribute Requirements:

- [] Inherit from `ValidationAttribute`
- [] Accept allowed categories in constructor
- [] Generate error message with allowed categories list
- [] Implement `IsValid()` method to check category against allowed list

PriceRangeAttribute Requirements:

- [] Inherit from `ValidationAttribute`

- ☐ Accept min and max price in constructor (as double, convert to decimal)
- ☐ Generate error message with currency formatting
- ☐ Implement `IsValid()` method for price range validation

Task 3.3: Implement Conditional Product Validation (7 minutes)

Conditional Validation Requirements:

- ☐ **Electronics Product Conditions:**
 - ☐ Price minimum \$50.00 (stricter than base rule)
 - ☐ Must contain technology keywords in Name
 - ☐ Must be released within last 5 years (cross-field validation)
- ☐ **Home Product Conditions:**
 - ☐ Price maximum \$200.00
 - ☐ Name must be appropriate for home products (no inappropriate words)
- ☐ **Clothing Product Conditions:**
 - ☐ Brand name minimum 3 characters
- ☐ **Cross-Field Validation:**
 - ☐ Expensive products (>\$100) must have limited stock (≤20 units)
 - ☐ Electronics must be recent (released within 5 years)

Helper Methods Required:

- ☐ `ContainTechnologyKeywords()`: Check Name against technology keywords list
 - ☐ `BeAppropriateForHome()`: Check Name against inappropriate words for home products
-

Module 4: Integration & Testing for Products (15 minutes)

Task 4.1: Update Program.cs Configuration (5 minutes)

Requirements:

- ☐ Register both AutoMapper profiles (`ProductMappingProfile`, `AdvancedProductMappingProfile`)
- ☐ Register `CreateProductProfileValidator` as scoped service
- ☐ Register all validators from assembly containing `CreateProductProfileValidator`
- ☐ Add `CorrelationMiddleware` to pipeline
- ☐ Update endpoint mapping to `/products` with product-specific documentation

Task 4.2: Create Product Integration Tests (10 minutes)

Test Class Requirements:

- [] Name: `CreateProductHandlerIntegrationTests`
- [] Implement `IDisposable`
- [] Set up in-memory database with unique name
- [] Configure AutoMapper with both product profiles
- [] Set up memory cache
- [] Mock `ILogger<CreateProductHandler>`
- [] Create handler instance with all dependencies

Test Methods Required:

- [] **Test 1:**
`Handle_ValidElectronicsProductRequest_CreatesProductWithCorrectMappings`
 - [] Arrange: Create valid Electronics product request with all properties
 - [] Act: Call handler
 - [] Assert: Verify Created result type
 - [] Assert: Check CategoryDisplayName = "Electronics & Technology"
 - [] Assert: Check BrandInitials for two-word brand
 - [] Assert: Check ProductAge calculation
 - [] Assert: Check FormattedPrice starts with currency symbol
 - [] Assert: Check AvailabilityStatus based on stock
 - [] Assert: Verify ProductCreationStarted log called once
- [] **Test 2:**
`Handle_DuplicateSKU_ThrowsValidationExceptionWithLogging`
 - [] Arrange: Create existing product in database with specific SKU
 - [] Arrange: Create request with same SKU
 - [] Act & Assert: Verify ValidationException thrown
 - [] Assert: Check exception message contains "already exists"
 - [] Assert: Verify ProductValidationFailed log called once
- [] **Test 3:**
`Handle_HomeProductRequest_AppliesDiscountAndConditionalMapping`
 - [] Arrange: Create valid Home product request
 - [] Act: Call handler
 - [] Assert: Check CategoryDisplayName = "Home & Garden"
 - [] Assert: Check Price has 10% discount applied
 - [] Assert: Check ImageUrl is null (content filtering)

Test Infrastructure Requirements:

- [] Proper disposal of context and cache
 - [] Unique database name per test run
 - [] Proper exception handling in tests
 - [] Mock verification for logging calls with product-specific events
-

Assessment & Deliverables Requirements

Expected Outputs (All Required):

- ☐ **Advanced AutoMapper Profile** with custom resolvers and conditional logic for products
- ☐ **Enhanced CreateProductHandler** with comprehensive logging and performance tracking for product operations
- ☐ **Complex FluentValidation** with async rules and product-specific business logic
- ☐ **Correlation Middleware** for request tracking
- ☐ **Integration Tests** demonstrating all product functionality

Assessment Criteria (Total: 100 points):

AutoMapper (25 points):

- ☐ **10 points:** Custom value resolvers implemented correctly
 - CategoryDisplayResolver with product-specific names
 - PriceFormatterResolver with currency formatting
 - ProductAgeResolver with product age logic
 - BrandInitialsResolver with proper name parsing
 - AvailabilityStatusResolver with stock-based status
- ☐ **10 points:** Conditional mapping logic works
 - Category-based ImageUrl filtering for home products
 - Category-based Price discount for home products
 - Mappings produce expected results for all categories
- ☐ **5 points:** Clean, maintainable mapping configuration
 - Well-organized product mapping profile classes
 - Clear method names and product-specific structure
 - Proper separation of concerns

Logging (25 points):

- ☐ **10 points:** Structured logging with correlation IDs
 - CorrelationMiddleware properly implemented
 - Product operation logging scopes created correctly
 - Correlation IDs flow through product requests
- ☐ **10 points:** Performance metrics captured
 - All product timing measurements implemented
 - ProductCreationMetrics properly populated with product data
 - Metrics logged in structured format with SKU, Name, Category
- ☐ **5 points:** Appropriate log levels and events
 - Correct LogLevel usage for product operations
 - Proper product-specific EventId constants used (2001-2008)
 - Meaningful log messages with product context

Validation (30 points):

- ☐ **15 points:** Complex async validation rules
 - SKU uniqueness checks working
 - Name+Brand uniqueness validation implemented
 - Product business rules validation implemented
 - Proper async/await usage with product context
 - Cancellation token support
- ☐ **10 points:** Product-specific business rule validation
 - Daily product addition limit check
 - Category-specific price rules (Electronics \$50+, Home \$200-)
 - Content filtering for home products
 - High-value product stock constraints
 - Electronics recency requirements
 - Proper error handling and logging
- ☐ **5 points:** Custom validation attributes
 - ValidSKUAttribute working with format validation
 - ProductCategoryAttribute working
 - PriceRangeAttribute working with currency
 - Client-side validation support

Integration (20 points):

- ☐ **10 points:** All components work together
 - Program.cs properly configured for product management
 - All product services registered correctly
 - Middleware pipeline correct for product operations
 - No runtime errors in product workflows
- ☐ **10 points:** Comprehensive test coverage
 - All three test methods pass
 - Proper mock verification with product-specific events
 - Product edge cases covered (categories, pricing, stock)
 - Clean test setup and disposal

Performance Requirements:

- ☐ **Sub-100ms average creation time** for successful product creation requests
- ☐ **Proper memory management** (no leaks in product tests)
- ☐ **Efficient database queries** (single query for SKU uniqueness checks)
- ☐ **Appropriate caching** (product cache invalidation working)

Code Quality Requirements:

- ☐ **All code files properly organized** for product management structure
- ☐ **Consistent naming conventions** throughout product domain
- ☐ **Proper error handling** (no unhandled exceptions in product operations)
- ☐ **Clean separation of concerns** (each product class has single responsibility)
- ☐ **Comprehensive XML documentation** on public product methods
- ☐ **No compiler warnings** or errors in product code

Bonus Challenges (Optional, +10 points each):

- ☐ **Category-based Caching:** Implement separate cache keys for different product categories
 - Different cache keys for Electronics, Clothing, etc.
 - Category-specific cache invalidation strategy
 - Performance improvement measurable
- ☐ **Product Metrics Dashboard:** Create endpoint to expose product creation and inventory metrics
 - New endpoint returning product aggregated metrics
 - Real-time product performance data
 - Proper DTO for product metrics response
- ☐ **Multi-language Product Support:** Support product Names and descriptions in multiple languages
 - Resource files for product metadata in multiple languages
 - Culture-specific product information
 - Proper fallback to default language
- ☐ **Inventory Optimization:** Implement batch product creation with stock management
 - Accept array of product requests
 - Batch product database operations
 - Transaction management for product inventory
 - Parallel processing where appropriate

Time Management Requirements:

- ☐ **Module 1 completed within 20 minutes** (product mapping)
- ☐ **Module 2 completed within 20 minutes** (product logging)
- ☐ **Module 3 completed within 25 minutes** (product validation)
- ☐ **Module 4 completed within 15 minutes** (product integration)
- ☐ **Total exercise completed within 80 minutes**

Success Metrics:

- ☐ **All product tests pass** without modification
 - ☐ **No runtime exceptions** during product operations
 - ☐ **Logging produces readable, structured output** for product workflows
 - ☐ **AutoMapper mappings work correctly** for all product scenarios
 - ☐ **Validation prevents invalid product data** from being saved
 - ☐ **Performance metrics show** product operation timing breakdown
 - ☐ **Correlation IDs appear** in all related product log entries
-

Final Checklist Before Submission:

Code Files Required:

- [] `Features/Products/ProductCategory.cs`
- [] `Features/Products/Product.cs` (updated with product properties)
- [] `Features/Products/DTOs/AdvancedProductDtos.cs`
- [] `Common/Mapping/AdvancedProductMappingProfile.cs`
- [] `Common/Logging/LoggingModels.cs` (updated for products)
- [] `Common/Logging/LoggingExtensions.cs` (updated for products)
- [] `Common/Middleware/CorrelationMiddleware.cs`
- [] `Validators/CreateProductProfileValidator.cs`
- [] `Validators/Attributes/ValidSKUAttribute.cs`
- [] `Validators/Attributes/ProductCategoryAttribute.cs`
- [] `Validators/Attributes/PriceRangeAttribute.cs`
- [] `Features/Products/CreateProductHandler.cs` (updated for products)
- [] `Program.cs` (updated for product endpoints)
- [] `Tests/CreateProductHandlerIntegrationTests.cs`

Verification Steps:

- [] **Build succeeds** without errors or warnings
- [] **All product tests pass** in test runner
- [] **API starts successfully** and responds to product requests
- [] **Swagger documentation** shows updated product endpoints
- [] **Product logs appear in console** with proper formatting
- [] **Product database operations work** (create, check SKU uniqueness)
- [] **Product caching behavior verified** (check cache invalidation)
- [] **Exception handling working** for product operations
- [] **Product category mapping works** for all categories
- [] **SKU validation works** for proper format